

ArSignTutor: An Interactive Real-Time Arabic Sign Language Learning System Using Deep Learning

Zeed Al Haj Ali
Bahcesehir University
Department of Artificial Intelligence
zeed.alhajali@bahcesehir.edu.tr

Abstract

Sign language is the primary mode of communication for millions of deaf and hard-of-hearing individuals worldwide. While American Sign Language (ASL) recognition has been studied extensively, Arabic Sign Language (ArSL) remains underexplored despite serving an estimated seven million Arabic-speaking deaf individuals. In this work, we present ArSignTutor, an interactive real-time learning system for the Arabic Sign Language alphabet. Our approach combines a fine-tuned MobileNetV2 convolutional neural network with MediaPipe-based hand detection for robust, real-time gesture classification, and pairs it with a spelling-based tutoring interface that guides users through word formation letter-by-letter with immediate visual feedback. Training MobileNetV2 on the ArASL dataset (over 54,000 images) under a random image-level split yields 99.26% test accuracy; we show, however, that this figure is inflated by data leakage, since the dataset contains many near-duplicate frames of the same signer that a random split distributes across both training and test sets. To obtain an honest estimate of generalization, we evaluate the model across datasets on an independent RGB Arabic alphabet dataset (31 shared classes), where accuracy falls to 48.78%, and then retrain under a leakage-aware (cluster-based) split with hand-region preprocessing, reaching a defensible 94.33% test accuracy. We further introduce a selective hand-aware background-blurring technique to improve the clarity of the real-time interface. Our main contribution is methodological: a reproducible protocol that separates genuine generalization from leakage-driven optimism, alongside a practical interactive tool for the Arabic-speaking deaf community.

1. Introduction

Background. Sign language is one of the most important communication methods [?] used by deaf and hard-of-

hearing individuals around the world. It lets people with hearing disabilities communicate using hand gestures, facial expressions, and body movements instead of spoken language. However, learning sign language can be hard for beginners because there are few accessible educational tools and interactive learning systems.

With the progress of artificial intelligence and computer vision, deep learning models can now recognize complex visual patterns such as hand gestures [?]. This makes it possible to build systems that recognize sign language automatically from camera input. Yet most existing systems focus on American Sign Language (ASL) [?], while Arabic Sign Language (ArSL) gets far less attention [?] and has fewer interactive educational applications, even though Arabic is among the most spoken languages in the world and its deaf community is large.

Beyond language coverage, research has mostly emphasized accuracy on benchmark datasets, without always asking whether that accuracy reflects real performance. A high number on a benchmark does not automatically make a useful tool: the score can be inflated by how the data is split and evaluated, and lab conditions differ from real-world use. Closing the gap between reported benchmarks and practical educational tools therefore needs attention not only to interaction and real-time performance, but also to whether the evaluation itself is valid.

1.1. Problem Statement

Although recent deep learning systems report strong performance in sign language recognition, most work focuses on ASL, while ArSL still suffers from limited datasets, fewer educational applications, and a lack of interactive real-time systems built for Arabic-speaking users.

Many existing systems also work only as static classifiers that predict a gesture without offering an engaging learning experience, with little support for interactive practice or real-time feedback.

A further, often ignored problem is the validity of the reported accuracy itself. Sign-language datasets are fre-

quently built from many near-duplicate frames of the same signer. When such data is split randomly at the image level, almost-identical frames land in both the training and test sets. This leaks information and inflates test accuracy, so the number no longer reflects how the model performs on new users. A high score obtained this way can hide poor real-world performance.

Finally, real deployment brings variation in lighting, hand position, camera quality, and background clutter that can lower performance outside the lab, which is especially clear in webcam-based systems.

There is therefore a need for an Arabic Sign Language learning platform that combines *honestly evaluated* recognition with an interactive tutoring experience that works under practical conditions.

1.2. Project Objectives

The main objective is to build an interactive real-time Arabic Sign Language learning system using deep learning and computer vision. The specific objectives are:

- Develop a real-time Arabic sign language recognition system using a fine-tuned MobileNetV2 model.
- Recognize the Arabic alphabet sign classes using the ArASL dataset.
- Establish an honest, leakage-free evaluation, and measure generalization on an independent dataset instead of trusting a single benchmark score.
- Build an interactive tutoring interface that guides users through word spelling letter-by-letter with immediate feedback.
- Improve robustness to real-world conditions such as background clutter and lighting changes.
- Deploy the system with a lightweight architecture suitable for real-time webcam use.

1.3. Project Contributions

The main contributions are:

- An interactive Arabic Sign Language learning system that combines deep learning recognition with educational tutoring.
- A real-time webcam-based recognition pipeline using a fine-tuned MobileNetV2 architecture.
- A word-based spelling tutor that guides users through sequential letter prediction and practice.
- Hand-focused preprocessing and background mitigation to improve robustness.
- **Identifying and quantifying data leakage** in the standard random split of frame-based ArSL datasets, and showing that a small train/validation gap under such a split is *not* evidence of generalization.
- A **leakage-aware (cluster-based) split and cross-dataset evaluation protocol** that gives a realistic generalization estimate instead of an inflated benchmark score.

2. Related Work

Sign language recognition has grown into an active research area in computer vision. Early methods relied on hand-crafted features and classical classifiers, but convolutional neural networks (CNNs) largely replaced them because they learn useful features directly from images [? ?]. Since training a large network from scratch needs a lot of data, most recent work uses transfer learning, where a model pre-trained on a large image dataset is adapted to the gesture task. Lightweight backbones such as MobileNet [4] and ResNet [?] are common choices, because they run fast enough for real-time use while still classifying accurately.

Most of this progress has gone to American Sign Language [?], which now has many public datasets and recognition systems. Arabic Sign Language has received far less attention [?], even though the need for Arabic learning tools is clear. Several studies have applied CNNs to Arabic alphabet recognition [? ?] and report high accuracy. Two limitations show up often: the systems are usually static classifiers with no teaching component, and the accuracy is measured on a single controlled dataset, which does not always reflect how the model behaves on new users or in different conditions.

2.1. Educational Sign Language Systems

Some work looks beyond pure recognition at how computer vision can help people learn. Tools that give immediate visual feedback tend to help beginners more than passive video tutorials, since the learner can fix mistakes as they happen. Most of these tools target ASL or European sign languages, so Arabic learners are left with few options. Combining accurate recognition with a structured learning task is still uncommon for Arabic, and that is the gap this work tries to fill.

2.2. Transfer Learning for Limited-Data Domains

Transfer learning is the usual choice when the target task has limited data. The low-level features a network learns on natural images, such as edges and textures, carry over well to hand images, so a pretrained backbone gives a strong starting point. A common recipe is two-phase fine-tuning: first train only the new classifier head while the backbone stays frozen, then unfreeze the whole network and continue at a smaller learning rate. Freezing the backbone early keeps the untrained head from corrupting the pretrained weights. We use this approach for the ArASL task.

3. Methodology

3.1. Datasets and Evaluation

We use the Arabic Sign Language dataset (ArASL) [?], a public dataset of over 54,000 grayscale images covering

the 32 Arabic alphabet signs, collected from more than 40 contributors under varied hand positions.

A practical issue with this dataset is that it contains many near-duplicate frames of the same person holding the same letter. If the images are split randomly, almost-identical frames end up in both the training and test sets, so the model can succeed on test images simply because it saw their near-twins during training. This is data leakage, and it makes test accuracy look much higher than the model’s real ability. Our first split was random, and the resulting accuracy turned out to be inflated for exactly this reason.

To fix this, we use a leakage-aware split. The dataset has no signer labels, so we approximate one: we compare images with a perceptual hash (dHash), group near-duplicate frames into clusters, and send each cluster as a whole to only one of the training, validation, or test sets (70/15/15). After clustering, the RGB images formed 7,575 clusters out of 7,856 images; assigning whole clusters gives 5,509 training, 1,182 validation, and 1,165 test images, with no cluster shared across any two splits. This guarantees that no frame and its near-copies can appear on both sides of the split.

To check generalization beyond one source, we also use a second, independent dataset: the RGB Arabic Alphabets Sign Language Dataset, which is in color and was collected separately. After matching the class names between the two datasets (they use different spellings, and the RGB set has no `yaa`, which we drop), we obtain 31 shared classes. We evaluate in two ways: a clean held-out test set from the same distribution, and a cross-dataset test on the independent set.

3.2. Preprocessing and Data Augmentation

All images are resized to 224×224 to match the MobileNetV2 input, turned into tensors, and normalized with the standard ImageNet statistics. We also focus the model on the hand region using MediaPipe-based detection and cropping, which removes much of the background variation that hurts performance on new data. The same preprocessing is used during training and during live inference, so the model sees images the same way in both cases.

During training we apply light augmentation to imitate real webcam conditions: small random rotations (up to about $\pm 10^\circ$), small affine shifts and scaling, and mild brightness and contrast changes. We deliberately do not use horizontal flipping, because mirroring a hand can turn one sign into a different one. Validation and test images are not augmented.

3.3. Model Architecture

We use MobileNetV2 [4] as the backbone. It is a good fit here because it is accurate while staying light enough to run in real time on a normal laptop. Instead of training from scratch, we start from ImageNet pretrained weights [1] and adapt them to our task. The final classification layer is

replaced to output the number of sign classes (32 for the ArASL model, and 31 for the domain-adapted RGB model after dropping `yaa`).

Training has two phases. First we train only the new classifier layer while the backbone is frozen, so the network can adjust to the new labels without damaging the pretrained features. Then we unfreeze the whole network and fine-tune it at a smaller learning rate. All training is done in PyTorch [3] on a Kaggle NVIDIA Tesla T4 GPU.

3.4. Domain Adaptation to Color Data

After the leakage analysis, we adapt the model to the color (RGB) distribution, which is closer to what a real webcam produces. We initialize from the ImageNet and ArASL weights and continue training on the RGB dataset using the leakage-aware split described above. Because the test set here is genuinely unseen, the resulting accuracy is a fair estimate of how the model performs on new data, rather than an inflated benchmark number.

3.5. Hand-Aware Background Blurring

To make the live interface easier to read, we blur the background around the detected hand. The steps are:

1. Detect 21 hand landmarks with MediaPipe Hands [2].
2. Build a soft elliptical mask M centered on the hand, with edges smoothed by a Gaussian blur of the mask.
3. Blur the whole frame with a Gaussian kernel (51×51) to get a blurred copy I_{blur} .
4. Blend the sharp and blurred frames using the mask:

$$I_{\text{out}} = I_{\text{sharp}} \cdot M + I_{\text{blur}} \cdot (1 - M) \quad (1)$$

The result keeps the hand sharp while softening the background, and because the mask edges are smoothed, the transition looks natural. The same idea could later be used to mask the background before classification, which we leave for future work.

3.6. Training Strategy

We train with the AdamW optimizer and cross-entropy loss with label smoothing ($\epsilon = 0.10$), which discourages overconfident predictions. The classifier head is trained for 5 epochs with the backbone frozen, then the full network is fine-tuned for up to 25 epochs at a lower learning rate. We use ReduceLROnPlateau to lower the learning rate when validation stops improving, mixed-precision training (AMP) for speed, and early stopping (patience 6). The best model on the validation set is kept. The full configuration is shown in Table ??.

Note on the loss values: with label smoothing, the cross-entropy loss has a non-zero floor (around 0.37 for this number of classes), so a loss near that value at high accuracy is expected and does not mean the model is underfitting.

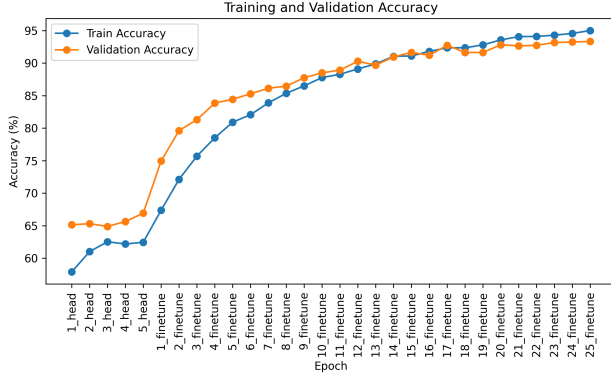


Figure 1. Training and validation accuracy across the warm-up (head) and fine-tuning epochs on the leakage-aware split. The gradual climb, rather than an instant jump, indicates genuine learning rather than leakage.

4. Experimental Results

4.1. Training Performance

We trained the model in two phases: a short warm-up where only the classifier head is updated, followed by full fine-tuning of the whole network at a smaller learning rate. The warm-up alone gave limited accuracy (around 67% validation), as expected, since most of the network was still frozen. Once we unfroze all layers, accuracy climbed steadily across the fine-tuning epochs.

On the leakage-aware split, training and validation accuracy rose together and stayed close, reaching about 95% training and 93.32% validation accuracy by the final epoch. The way they rise matters: instead of jumping to a near-perfect score in the first epoch, the curves climb gradually, which is what genuine learning looks like. This is the opposite of our first (leaky) run, where validation accuracy was already near the top from the start because near-duplicate frames were shared between training and test. Figure 1 shows the accuracy curve and Figure 2 the corresponding loss.

4.2. Evaluation Across Regimes: The Data Leakage Analysis

Our first model reached 99.26% test accuracy, but this number came from a random image-level split and is inflated by leakage. To make this concrete, we compare three very different evaluation settings in Table 1.

We first ran a sanity check: using the exact inference pipeline, the original model scored 98.75% on a small ArASL sample. This only tells us the pipeline (model loading, class order, preprocessing) is correct, not that the model generalizes.

The real test is the cross-dataset evaluation. When we ran the ArASL-trained model on the independent RGB dataset,

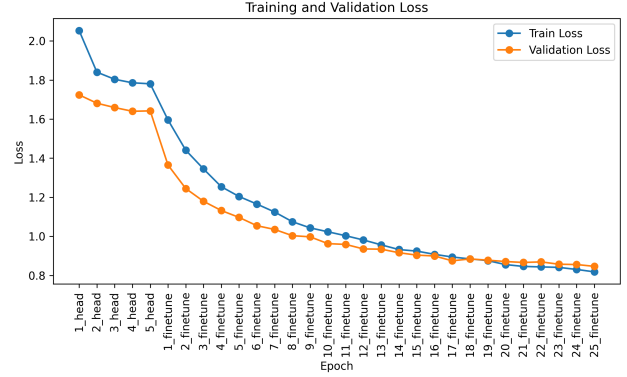


Figure 2. Training and validation loss over the same epochs. Both decrease smoothly; label smoothing keeps the loss from reaching zero.

Table 1. Accuracy under different evaluation settings. The sanity check only confirms that the inference pipeline is correct; it is not a generalization measure.

Setting	Split	Test set	Acc.
Original model	Random (image)	ArASL	99.26%
Sanity check	—	ArASL	98.75%
Cross-dataset	Train on ArASL	RGB (31 cls.)	48.78%
Clean retrain	Cluster-based	Held-out	94.33%

accuracy dropped to 48.78%. That is still well above random chance (about 3% for 31 classes), so the model did learn real features, but it is far from the 99.26% it reported on its own dataset. Looking at the per-class scores, the pair τ_a/τ_{aa} scored zero, which points to a label-matching or sign-convention mismatch for that pair rather than a model failure; setting it aside raises the figure to about 52%.

The drop from 99.26% to 48.78% should not be blamed entirely on leakage. It mixes three things: real data leakage, a domain shift (different people, lighting, cameras, and backgrounds), and the class mismatch above. Retraining with the leakage-aware split removes the leakage part and gives the honest 94.33% reported in Table 1.

4.3. Quantitative Results

Table 2 summarizes the final model. After retraining on the color data with the clean split, the model reaches 94.33% on a held-out test set, with a weighted F1-score of 94.36%. This is the accuracy we report as a fair estimate of its real performance.

Figure 3 shows the confusion matrix on the clean test set. Most of the weight is on the diagonal, and the errors that remain fall between letters with very similar hand shapes, which is the natural limit of recognizing a single still image. Table 3 breaks the accuracy down by class. Ev-

Table 2. Final results of the leakage-aware ArSignTutor model on the clean held-out test set.

Metric	Value
Test Accuracy	94.33%
Validation Accuracy	93.32%
Precision (weighted)	94.59%
Recall (weighted)	94.33%
F1-score (weighted)	94.36%
Cross-dataset (prior model)	48.78%
Number of Classes	31
Architecture	MobileNetV2
Input Resolution	224 × 224
Optimizer	AdamW

ery class reaches an F1-score above 0.85, and no class collapses, which shows the model learned all 31 letters rather than trading some off against others.

4.4. Analysis of Generalization Performance

One of the most important findings of this work is the difference between benchmark accuracy and true generalization performance. The initial MobileNetV2 model achieved more than 99

This result highlights the importance of evaluating sign language recognition systems under realistic conditions. High performance on a single dataset does not necessarily guarantee strong generalization to new users, different cameras, lighting conditions, or collection environments.

After applying the leakage-aware cluster-based splitting strategy and performing domain adaptation on the RGB dataset, the final model achieved 94.33

These findings suggest that careful dataset design and evaluation protocols are as important as model architecture when developing practical sign language recognition systems.

4.5. Analysis of Generalization Performance

One of the most important findings of this work is the difference between benchmark accuracy and true generalization performance...

...

4.6. Real-Time System Evaluation

We put the trained model into a real-time webcam app built with Streamlit and OpenCV. The user signs a letter in front of the camera and gets an immediate prediction. To help the model focus on the gesture, we crop to the hand region before classifying, which cuts down on background distractions. In testing, the system worked well under normal conditions, though accuracy still depends on lighting,

Table 3. Per-class precision, recall, and F1-score on the clean test set.

Class	Precision	Recall	F1
ain	0.97	0.97	0.97
al	0.95	0.95	0.95
aleff	0.98	1.00	0.99
bb	0.98	0.93	0.95
dal	0.80	0.91	0.85
dha	1.00	0.85	0.92
dhad	0.97	0.92	0.95
fa	0.92	0.87	0.89
gaaf	1.00	0.97	0.98
ghain	0.97	0.97	0.97
ha	1.00	1.00	1.00
haa	1.00	0.89	0.94
jeem	0.94	0.97	0.95
kaaf	0.85	0.87	0.86
khaa	0.95	0.97	0.96
la	0.98	0.98	0.98
laam	0.90	0.97	0.94
meem	0.97	0.97	0.97
nun	1.00	0.89	0.94
ra	0.83	0.97	0.90
saad	0.95	0.95	0.95
seen	0.93	0.95	0.94
sheen	0.98	0.98	0.98
ta	0.94	0.96	0.95
taa	0.93	0.88	0.90
thaa	0.94	0.96	0.95
thal	0.87	0.90	0.89
toot	0.97	0.93	0.95
waw	0.97	0.94	0.96
ya	0.90	1.00	0.95
zay	1.00	0.93	0.96
Weighted avg	0.95	0.94	0.94

hand position, camera quality, and how similar two letters look.

On top of single-letter prediction, the app includes a spelling tutor that walks the user through Arabic words one letter at a time. This turns the project from a plain classifier into something a learner can actually practice with.

5. Discussion

5.1. Training and Evaluation

The leakage-aware model trained smoothly, with training and validation accuracy rising together and no sign of overfitting. It is worth being clear about how we read this. In our first run, validation accuracy jumped above 99% within a few epochs and tracked training accuracy almost exactly, and we first took the tiny gap as proof of strong generalization. The cross-dataset analysis showed that reading was

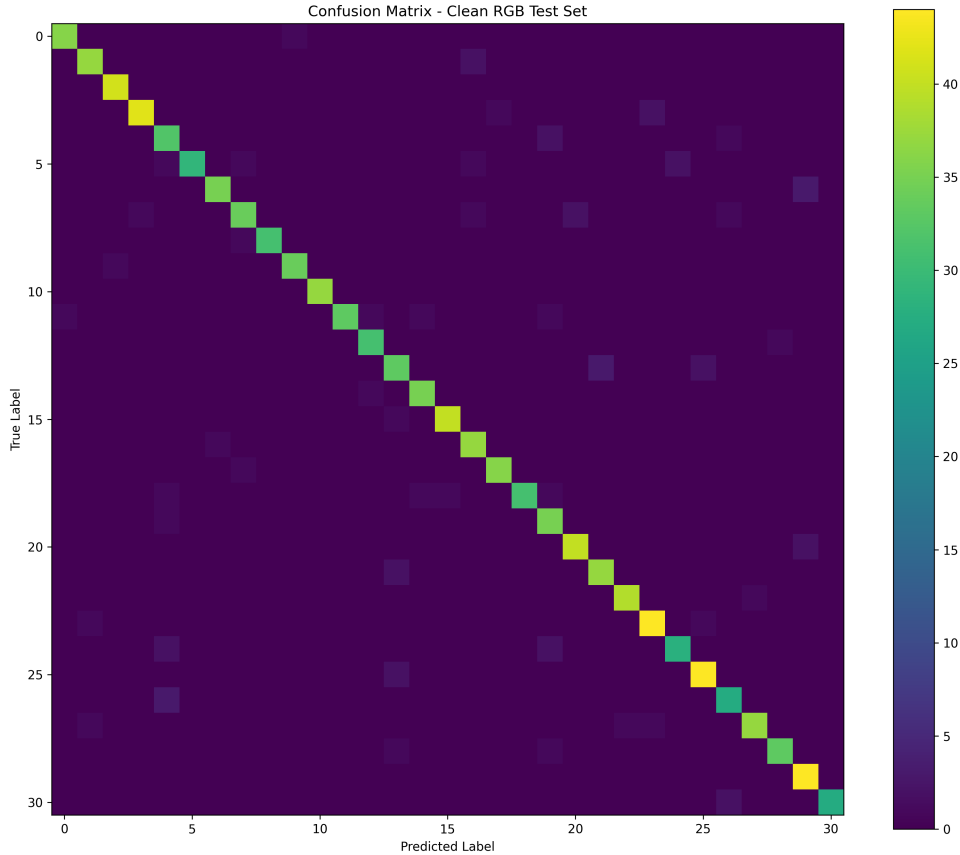


Figure 3. Confusion matrix on the clean held-out test set. The strong diagonal shows correct classification across the 31 classes; the few off-diagonal errors fall between letters with similar hand shapes (e.g., `dal` and `thal`, which differ only by a dot).

wrong. Under a leaky random split, a small gap is actually expected, because near-duplicate frames sit on both sides of the split, so the validation set is not really new to the model. Only after switching to the leakage-aware split does a small gap mean what we want it to mean, and there the model reaches 94.33% on a genuinely unseen test set. The label smoothing we used ($\epsilon = 0.10$) also keeps the loss from reaching zero, which is why the loss settles near 0.37 even at high accuracy.

5.2. Per-Class Performance

Accuracy is high and even across the alphabet: every class reaches an F1-score above 0.85, and none collapses. The weaker classes are `kaaf`, `dal`, and `thal`, with `fa` and `ra` slightly behind the rest. These are the letters whose hand shapes look almost the same and differ only in small details. A clear example is `dal` and `thal`, which differ only by a dot. Such cases are hard for any system that works from a single still image, because the distinguishing detail is small and not always visible.

5.3. Misclassification Analysis

Looking at the wrong predictions, most errors happen between letters with very similar hand shapes rather than between random pairs, which matches the per-class results above. The model is also less confident on these cases, with lower probability scores than on correct predictions. That suggests a simple improvement for future versions: when confidence is low, the system could ask the user to repeat the sign instead of guessing.

One earlier case is worth noting. In the cross-dataset test, the pair `ta`/`taa` failed completely, which traced back to a difference in how the two datasets label or sign that pair, not to a weakness in the model. After retraining within a single, consistent dataset, both letters are recognized well. It is a good reminder that matching letters by name across datasets is not the same as matching the actual gesture.

5.4. Strengths of the Approach

A few choices helped the system work well:

- Starting from ImageNet pretrained weights gave a strong base, so the model adapted to hand images with limited

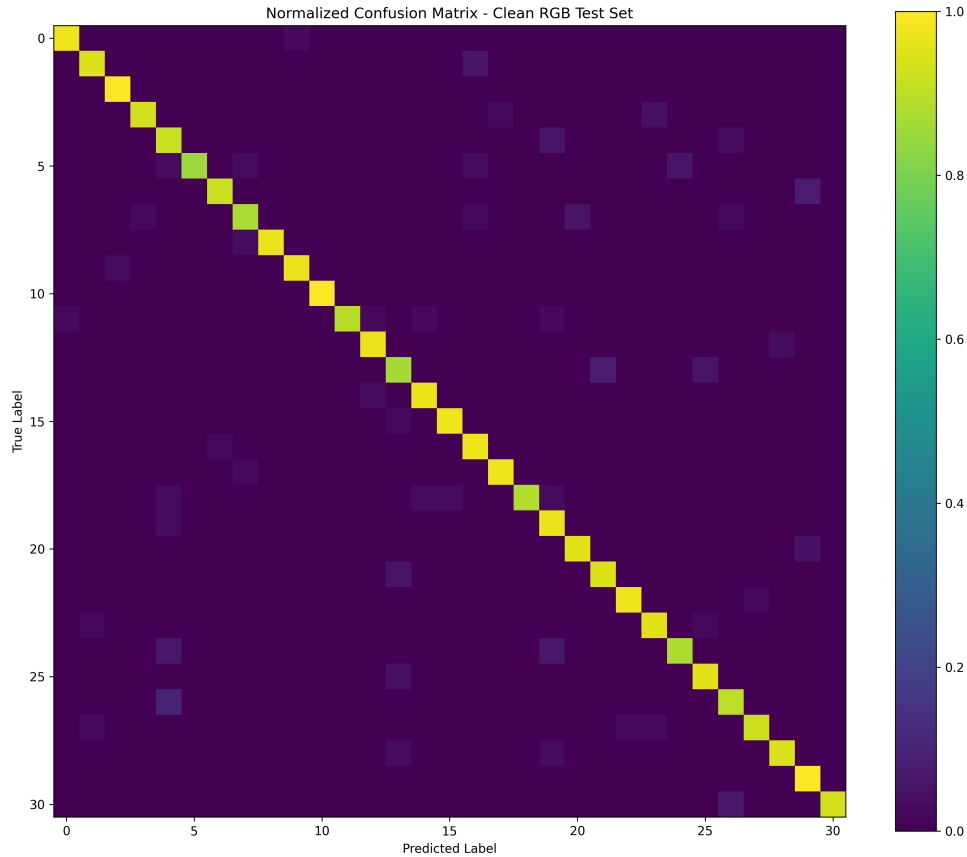


Figure 4. Normalized confusion matrix showing class-wise recognition performance on the clean RGB test set.

data.

- The two-phase training kept the pretrained features intact early on and let the network adjust gradually.
- MobileNetV2 is light enough to run in real time on a normal laptop, with no special hardware.
- The spelling tutor turns the model into something a learner can practice with, not just a classifier.
- Most importantly, we evaluated the model honestly. Instead of reporting a single inflated benchmark number, we found the leakage, measured the model on a separate dataset, and retrained so that the reported 94.33% reflects real performance.

5.5. Limitations

Several limitations remain:

- **Static letters only.** The system recognizes single letter signs, not dynamic word-level signs that involve motion. Those would need a temporal model.
- **Similar letter pairs.** Letters that differ only by a small detail stay the main source of errors and may need extra cues to separate.
- **Still a domain gap.** Even after retraining, accuracy

across a fully different dataset is lower than within one dataset. Real users will vary in ways no single dataset fully covers.

- **One class dropped.** The color dataset has no *yaa*, so the retrained model covers 31 of the 32 letters. Adding it back needs more data.
- **Single-frame decisions.** Each frame is classified on its own, which can make predictions jump while the user moves between signs. Averaging over a few frames helps but does not fully fix it.

5.6. Broader Impact

This work targets a group with few tools available to them: Arabic-speaking deaf and hard-of-hearing learners. By giving immediate feedback during letter-spelling practice, Ar-SignTutor makes self-directed learning easier, and it can help early learners, parents of deaf children, and teachers in places without specialized instructors. Just as important, we hope the leakage analysis is a useful reminder for similar projects: a near-perfect score is not always a good model, and it is worth checking how the data was split before trusting the number.

References

- [1] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255, 2009. [3](#)
- [2] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuoling Chang, Ming Guang Yong, Juhyun Lee, et al. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019. [3](#)
- [3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019. [3](#)
- [4] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018. [2](#), [3](#)